Lecture 4 in-camera

COMP30022, Semester 2, 2025

Running example

- Continue to develop an app for managing recipes online

- *Can you please give an architecture for an app for recipe management including recipe creation, recipe editing, and recipe search*

Architecture (again using Spark)

Certainly! I'll outline a high-level architecture for a recipe management app that includes recipe creation, editing, and search functionalities. This architecture will be based on a modern, scalable, and maintainable approach.

Recipe Management App Architecture

1 Overall Architecture

2 Components: Client Applications; API Gateway; Microservices; Databases; Storage

3 Detailed Architecture

4 Service Description: User Service; Recipe Service; Search Service; Media Service

5 Key Processes: Recipe Creation; Recipe Editing; Recipe Search

6 Additional Considerations: Caching; Security; Scalability; Monitoring and Logging

Overall architecture

We'll use a microservices-based architecture with a client-server model. This approach allows for better scalability, easier maintenance, and independent deployment of services.

[Client Applications] <--> [API Gateway] <--> [Microservices] <--> [Databases]

more ...

What is a microservices-based architecture with a client-server model?

- A microservices-based architecture with a client-server model is a modern approach to designing and building complex software applications. Let me break this down for you:
- **Microservices-based Architecture**
- **Definition**: Microservices architecture is an approach to developing a single application as a suite of small, independently deployable services, each running in its own process and communicating with lightweight mechanisms.

...

Continued dialogue

My menu planning application is simple. Should I still use a micro-services architecture?

For a simple menu planning application, a microservices architecture might be overkill. While microservices offer numerous benefits for large, complex systems, they also introduce additional complexity that may not be justified for smaller applications. Let's consider the pros and cons for your situation:

- **Reasons to Avoid Microservices for a Simple App**

Increased Complexity:

...

Alternative Approach: Monolithic Architecture

Proposed Simple Architecture

Here's a simplified architecture that might be more suitable for your menu planning application:

[Client (Web/Mobile App)] v

[API Layer (Express.js)] v

[Application Logic (Node.js)] v

[Database (e.g., PostgreSQL)]

This architecture:

Uses a single backend application that handles all logic.

Has a clear separation between the client, API layer, application logic, and database.

Is easier to develop, deploy, and maintain for a small team or individual.

Can still be scaled by adding more powerful servers or implementing load balancing.

Is the dialogue helpful?

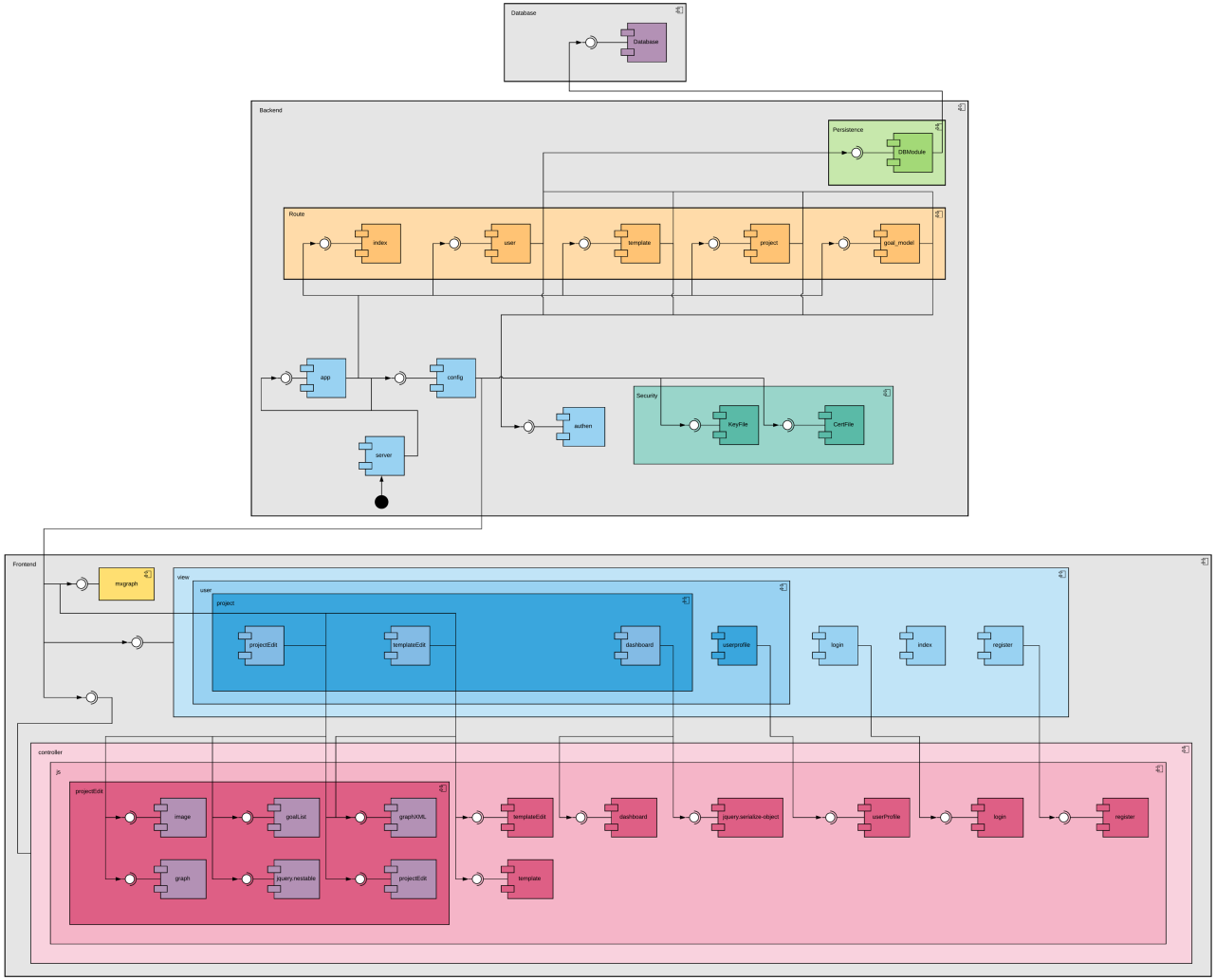
Improved interaction with an LLM

- Prompt engineering
  - Makes a difference, but it is ‘gaming the LLM’

- Simulating reasoning
  - RAG
  - Chain of Thought

- **Master a skill before using something you don’t understand**

MME design


Database
Database
Backend
Persistence
DBModule
Route
index
user
template
project
goal_model
app
config
server
authen
Security
KeyFile
CertFile
Frontend
magraph
view
user
project
projectEdit
templateEdit
dashboard
userprofile
login
index
register
controller
js
projectEdit
image
goalList
graphXML
graph
jquery.nestable
projectEdit
templateEdit
dashboard
jquery.serialize-object
userProfile
login
register
template